

AI Usage Journal

CSE 341 Semester Project — Recipix v4.1 (Lexer + Integration)

39 entries across 4 Claude sessions, May 4–8 2026

Overview

Four Claude sessions, four different roles. A 'professor' session in Claude.ai (web) graded each version of the spec against the handout's rubric. The main developer session in Claude Code (Sonnet 4.6) carried the implementation work over five days, including a domain pivot from DataScript to Recipix. Two later web sessions ran orthogonal reviews — a tester agent that ran 66+ independent test cases against the lexer/parser, and a code reviewer that surfaced Part-2-risk issues the tester missed. Most professor and reviewer turns were one-shot; the developer session was 54 raw turns consolidated below into 10 thematic entries.

Session 1 — Professor (Claude.ai Web)

Source: *Yeni_Metin_Belgesi.txt*

A Claude.ai web session where I asked the model to play the role of the professor who wrote the project handout. Used to grade my own design choices, push back on weak rationale, and keep me honest about whether each version of the spec was exam-defensible. May 7, 2026 — fourteen substantive turns.

Entry # 1 of 39 • Date 2026-05-07 • Phase Role setup • AI tool Claude.ai web (Sonnet, presumed)

Goal.

Set up a 'professor' reviewer role: a Claude that grades whatever I bring back from the other Claude (the implementer) against the project handout's rubric.

Prompt.

"these pdfs are my programming language homework i want u act like a professor who prepare this pdf and i ll use another chat (Claude code) but in this chat model gonna be sonnet... after each my answer i will ask you to this answer compatible with your conditions" — attached the handout PDF and the submission guide PDF.

Accepted.

It read both PDFs and confirmed the role with sharp boundaries: said it would (a) review against the §5 rubric, (b) check D1↔D2 consistency, (c) flag where Sebesta terminology is misused, (d) tell me when something reads like AI boilerplate that won't survive the 14 May exam, (e) quiz me. It also flagged §6 / §9 of the handout out loud — that AI-generated rationale I can't defend in the exam is treated as exam failure that propagates back to the artifact grade. That set the right tone.

Rejected / modified.

It refused to help me 'fake the AI Usage Journal or pretend work was yours when it wasn't.' I hadn't asked for that, but having it stated up front killed any temptation later in the project.

Errors caught.

None — it correctly read the rubric percentages and the deadline structure.

Reflection.

Asking the AI to play the strict reviewer rather than the helpful collaborator gives more useful feedback. It costs zero extra tokens and totally changes the outputs.

Entry # 2 of 39 • Date 2026-05-07 • Phase Spec review (v1) • AI tool Claude.ai web (Sonnet)

Goal.

Have the 'professor' read the first version of the Recipix spec my partner and I drafted and tell us what's broken before we start coding.

Prompt.

"these are the rules of our pl i forgot to say we are doing this with my friend so ours is pair homework not solo" — pasted the full Recipix v1 spec.

Accepted.

Said the dimensional type system is a real DSL choice (originality dimension covered), the structured-type story holds, and the three operations (evaluate/scale/substitute) give exam material. Said the §2 minimum requirements look met if I tighten primitive-type vocabulary.

Rejected / modified.

Pushed back hard on five things that I would have shipped without noticing: (1) primitive-type story not declared explicitly (§4.5 demands it); (2) "structural equivalence for dimensions, name equivalence for recipes" needs to be precisely defended in Sebesta §6.14 terms; (3) the spec says "no conditionals or loops" but the handout requires at least one control structure beyond sequencing — direct rule conflict; (4) errors like "negative scale value" exist but the language has no `<` or bool — I needed an answer for how that error gets raised; (5) `at` and `for` in step modifiers risk EBNF ambiguity.

Errors caught.

The control-structure conflict (#3 above) was the real save — handout §2 explicitly requires "at least one control structure beyond sequencing (if, while, for, or pattern-match)" and we had nothing. Would have lost points without this.

Reflection.

AI is good at cross-referencing two documents (handout vs spec) and finding consistency violations. This is exactly the kind of mechanical-but-tedious check that I miss when I'm tired.

Entry # 3 of 39 • Date 2026-05-07 • Phase Scope narrowing • AI tool Claude.ai web (Sonnet)

Goal.

Get a clean Part 1 vs Part 2 breakdown so I stop wasting time on Part 2 work.

Prompt.

"focus only part 1 for now ,part2 after we done part 1"

Accepted.

Produced a deliverables table for Part 1 (D1 §§4.1/4.2/4.3/4.6 only, D2 lexer+parser, D3 test report, D4 journal w/ E1, D5 retro, D6 contribution) and an explicit "NOT in Part 1" list (type checker, interpreter, §§4.4/4.5/4.7/4.8, executed outputs, type-error tests, E2/E3).

Rejected / modified.

Nothing rejected.

Errors caught.

None.

Reflection.

The "what NOT to do" list was as useful as the "what to do" list. Without it I had been mentally including Part 2 work in my time estimate.

Entry # 4 of 39 • Date 2026-05-07 • Phase Spec review (v2) • AI tool Claude.ai web (Sonnet)

Goal.

Re-review the spec after we addressed the v1 issues.

Prompt.

"now this is updated our decision check it out for part 1 and tell me there is any issue to design your homework" — pasted v2.

Accepted.

Said "much stronger spec than v1. The control-flow problem is fixed, the type story is clearer." Confirmed the §2 minimum-requirement gap was closed (we now had repeat/foreach loops + if/else).

Rejected / modified.

Still flagged: (1) keyword name clashes — `for` is both a step modifier and a loop keyword in some places, must disambiguate; (2) the EBNF for `step "... [at ...] [for ...] { ... }` is structurally ambiguous because `serves <expr>` parses greedily up to `{`; (3) primitive-type set still not totally explicit — write it as a literal table; (4) the action verb list (`combine, mix, pour, ...`) was 12 entries — they must all be reserved keywords or the EBNF breaks.

Errors caught.

The action-verb-as-keyword issue was a real catch — without it, a user could write `ingredient combine : 1 g` and the parser would accept it.

Reflection.

Iterating spec → review → fix → re-review three times in the same chat compounds the quality. Each pass surfaces problems the previous pass was too zoomed-out to see.

Entry # 5 of 39 • Date 2026-05-07 • Phase Roadmap review • AI tool Claude.ai web (Sonnet)

Goal.

Review the work-split roadmap I wrote (who does what, in which order, by which date).

Prompt.

"this is the our roadmap for complete each parts check this and detect the potential design issues of our pl"

Accepted.

Approved the lexer-then-parser dependency ordering. Flagged that I had Part 2 design questions mixed into Part 1 — said "since you said focus on Part 1 only, I'll review Part 1 design issues primarily, with brief notes on Part 2 only where Part 1 decisions lock in Part 2 problems."

Rejected / modified.

Said two things were ordering mistakes: (1) writing the EBNF *after* writing the parser is backwards — the EBNF is the contract, the parser is the implementation; (2) the journal entries were planned as "write all on May 8" — handout explicitly requires entries to be spread over the project, not all on the deadline.

Errors caught.

Caught the journal-dating rule. I had been planning to write all D4 entries on the last day; that would have failed the handout's 'real dates' requirement.

Reflection.

AI is unusually good at pointing out workflow ordering mistakes. The 'EBNF after parser' inversion is the kind of thing that's obvious once stated but invisible from inside the plan.

Entry # 6 of 39 • Date 2026-05-07 • Phase Pure language design review • AI tool Claude.ai web (Sonnet)

Goal.

Strip away workflow/process discussion and give me only the language design issues.

Prompt.

"for now dont care the task disturbition just focus pl design issue"

Accepted.

Produced a focused 11-issue list: (1) missing user-defined function support, (2) the substitute rule's scoping contradiction, (3) zero-arg call ambiguity (recipe vs function), (4) `quantity\_of` operator vs function-call confusion, (5) trailing comma policy, (6) `pinch` count ambiguity (`5 pinch` vs `1 pinch`), (7) precedence levels need explicit numbering, (8) error-message format requirement, (9) reserved-keyword list completeness, (10) string escape policy, (11) UTF-8 in identifiers.

Rejected / modified.

Nothing rejected.

Errors caught.

Issue #2 — the substitute scoping contradiction — was the most consequential. The spec said "the replacement ingredient name must be defined in scope at the substitute call site," but `substitute` is itself call-site-only (outside the recipe), so the replacement name is *not* in lexical scope at the call site. This had to be resolved as a special resolution rule (lookup against the recipe argument's ingredient table, not the lexical scope chain). The professor flagged this; the partner's parser team would have hit it as a bug otherwise.

Reflection.

Asking for "design issues only, not process" was a useful single-axis prompt. The more focused the question, the more focused the output.

Entry # 7 of 39 • Date 2026-05-07 • Phase Spec review (v3) • AI tool Claude.ai web (Sonnet)

Goal.

Re-review v3 after addressing the 11 design issues.

Prompt.

"Updated V3 specifications and roadmaps for each part"

Accepted.

"v3 is significantly stronger. The §2 minimum-requirement gap is closed, the keyword clashes are resolved, the syntax is unified."

Rejected / modified.

Still flagged a small but real list — issue #8 (`quantity\_of` should be operator-form not call-form to avoid the recipe-vs-function-call ambiguity), #10 (need to commit to a substitute-slot resolution rule explicitly), #11 (need a single canonical EBNF, not three different ones scattered across sections).

Errors caught.

The `quantity\_of` issue (#8) — keeping it as a call-form would have created exactly the same disambiguation problem as `r()` calls. Resolving it as a unary operator at precedence level 1 was the right call and it stuck through to the final implementation.

Reflection.

v3 took ~30 min to produce and was reviewed in 5. The review-driven loop is faster than 'write perfectly the first time' even though it sounds wasteful.

Entry # 8 of 39 • Date 2026-05-07 • Phase Readiness check • AI tool Claude.ai web (Sonnet)

Goal.

Yes/no — is v3 ready to start coding from?

Prompt.

"so this version is compatible with our homework and we go to build phase ?"

Accepted.

"Short answer: no, not yet" — direct refusal, no hedging.

Rejected / modified.

Said v3 was close but "still real design holes that will cost you points if you start coding now. Two more passes needed." Listed the three outstanding decisions to lock down before code: #8 (quantity_of as operator), #10 (substitute slot resolution rule), #11 (single canonical EBNF).

Errors caught.

None.

Reflection.

Asking a yes/no readiness question and getting a clean "no" is more useful than getting a long hedged answer. The model could have said "mostly yes, but..." and I would have started coding. "No, not yet" stopped me.

Entry # 9 of 39 • Date 2026-05-07 • Phase Spec generation (v4) • AI tool Claude.ai web (Sonnet)

Goal.

Have the model produce the v4 spec file with all the previously-discussed fixes applied.

Prompt.

"ok i want you to create file correspond these issue fixed pl design file like i uploaded before create v4 file fixed all issues"

Accepted.

It checked the docx skill (a sensible 'before doing it, look at the relevant skill' step) and produced a v4 spec markdown file with the fixes applied. The output stopped before the final v4.1 polish — that came in the next turn.

Rejected / modified.

Nothing rejected.

Errors caught.

None at this stage.

Reflection.

Watching the model think "let me check the relevant skill before generating a substantive document" was reassuring — it didn't just dive in.

Entry # 10 of 39 • Date 2026-05-07 • Phase Final design lock-in • AI tool Claude.ai web (Sonnet)

Goal.

Get a sign-off on the final three contested decisions (#8, #10, #11) before writing v4.1.

Prompt.

"So the real list of 'must decide before Part 1' is: #8 — quantity_of as operator (recommend yes)..." — pasted my proposed resolutions.

Accepted.

"Yes — these three decisions are all correct and exam-defensible. I'll apply them and produce v4.1." Then signed off on each one with a one-paragraph defense I could literally use in the May 14 written exam.

Rejected / modified.

Nothing rejected.

Errors caught.

None.

Reflection.

Asking the model to produce the exam answer alongside the design choice is two-for-one. The reasoning needed for the spec is the same reasoning needed for the exam — capturing it once saves a redo later.

Entry # 11 of 39 • Date 2026-05-07 • Phase Code review (initial) • AI tool Claude.ai web (Sonnet)

Goal.

Have the professor read the lexer + parser my partner and I built and check it matches the v4.1 spec.

Prompt.

"ok we developed lexer and parser and i ll uplaod these files to check it is compatible our design or any other issue check these files if there is any missed files tell me if not check our pl consistent with our design" — uploaded all source files.

Accepted.

Walked through every file. Confirmed: tokens.py covers all keyword sets correctly, the lexer's two-token quantity rule is implemented, ``C` is one token, parser implements the spec's precedence levels, AST node shapes match spec §14 exactly (QuantityOf operator-form, SubstituteCall with `str` slots).

Rejected / modified.

Flagged the 200g lexer bug — `200g` glued together was being accepted because the lexer's `\_scan\_number()` swallowed whitespace before checking for the missing separator. Spec Decision #7 says this must be a lexer error. Real bug.

Errors caught.

The 200g bug was a real find. I had not tested the no-separator case.

Reflection.

When you ask the model to review against a specific spec decision, it goes back and reads the decision before grading. That focus is what makes review-by-AI useful — without the spec, the model would just say "looks reasonable."

Entry # 12 of 39 • Date 2026-05-07 • Phase Architecture explanation • AI tool Claude.ai web (Sonnet)

Goal.

Confirm I understood the lexer→parser→AST pipeline well enough to defend it on May 14.

Prompt.

"system should be work like that there is recipe file consist some recipe and lexer reads this file and create tokens and parser get this token and check it is syntactically correct"

Accepted.

Confirmed my mental model was right: "Yes — that's exactly the architecture, and it's what your code already does." Then walked the end-to-end flow with concrete examples (a `.rcx`` source file → token stream → parser consumes tokens → AST → `--dump-ast`` printable output) and tied each stage to handout §4 D2 [P1] requirements.

Rejected / modified.

Nothing rejected.

Errors caught.

Caught a small misuse of vocabulary — I had been calling parser errors "lexical errors" interchangeably. The model corrected: lexer errors = unknown characters, parser errors = unexpected token sequences, *neither* is a "syntax error" in the loose sense, both are. Sebesta uses each precisely.

Reflection.

Vocabulary precision matters for the 14 May exam. The model corrected me without making it feel like a correction. Useful.

Entry # 13 of 39 • Date 2026-05-08 • Phase Re-review after 200g fix • AI tool Claude.ai web (Sonnet)

Goal.

Verify the 200g lexer fix is correct.

Prompt.

"i fixed 200g bugs check again whole project is ok now with this new lexer" — pasted the updated lexer.

Accepted.

Set up a fresh test environment, ran the same test battery as before plus new tests for the 200g case. Verified the fix: ``200g`` and ``1.5kg`` now correctly raise `LexerError`; ``200 g``, ``1.5 kg``, and ``200 /* comment */ g`` correctly produce two tokens.

Rejected / modified.

Pointed out one cosmetic note — there was an extra blank line at line 34 of the file. Acknowledged it was style not bug. Did not require a fix.

Errors caught.

No new errors. Confirmed the fix held.

Reflection.

Re-running the same tests after a fix takes 10 seconds for the model and gives me real confidence. Doing it manually would take 5 minutes and I'd skip half.

Entry # 14 of 39 • Date 2026-05-08 • Phase Final scope clarification • AI tool Claude.ai web (Sonnet)

Goal.

Make sure no logical bugs remain — separate cosmetic from substantive.

Prompt.

"so just the bug is message appearing not any logical error at code"

Accepted.

Crisp four-bucket breakdown: (1) logical/spec-correctness errors — none, zero remaining; (2) error-message wording — the only outstanding nit; (3) test coverage — 60+ tests passing; (4) integration — fully wired. Was direct: "Correct."

Rejected / modified.

Nothing rejected.

Errors caught.

None.

Reflection.

Closing a multi-day review session with a one-word "correct" plus a structured breakdown is the cleanest possible exit. I knew exactly what state the project was in.

Session 2 — Developer (Claude Code, Sonnet 4.6)

Source: `c9082500-6a74-4eb2-9786-0469e4982efb.jsonl`

My main implementation session in Claude Code, running across May 4–8. Started in Turkish, switched to English partway through (the 'lecture language is english' moment). 54 raw turns; consolidated below into 10 thematic entries because most were quick back-and-forth (paste an error, get a one-line fix, repeat). Owned the lexer; partner owned the parser; integration was joint.

Entry # 15 of 39 • Date 2026-05-04 • Phase Project setup / domain selection • AI tool Claude Sonnet 4.6 (Claude Code)

Goal.

Read the handout and pick a DSL domain.

Prompt.

"project handout pdfini oku part 1 için neler yapmma gerekli bana bir liste yap" — read the handout PDF, give me a Part-1 to-do list. Then over a few quick turns: discussed domain choices, eventually picked QueryLite (CSV query language).

Accepted.

Produced a clean Turkish-language to-do list with deliverable tables, deadlines (8 May 23:59), and dimension breakdown. Explained the four Sebesta evaluation criteria (readability/writability/reliability/cost) and how to defend them in the exam. When I narrowed to MusicScript or QueryLite, gave honest comparison: QueryLite has richer expression-language material but heavier Part 2 implementation; MusicScript is easier to implement but expression-language is thinner.

Rejected / modified.

I picked QueryLite based on the comparison; the model went along.

Errors caught.

None at this stage.

Reflection.

Letting the model explain *why* one option is better for which axis (exam material vs. implementation cost) was more useful than asking it to just recommend one.

Entry # 16 of 39 • Date 2026-05-04 • Phase QueryLite spec writing • AI tool Claude Sonnet 4.6

Goal.

Draft D1 §§4.1, 4.2, 4.3, 4.6 for QueryLite (a CSV query DSL).

Prompt.

Multiple short turns: "tamam hadi sırayla gidelim ama sınava gireceğimi unutma neyi neden yaptığını neye dayanarak yaptığını neden yapmak istediğini bana söyle" (go step by step, but remember I'm sitting an exam — tell me what you're doing and why), "Querylite ile devam edelim", "dataScript", "evet hadi yapalım", "evet", "devam".

Accepted.

Wrote Sebesta-grounded justifications for every design choice. Built D1_design_spec.md with all four required Part-1 sections. Got the EBNF, lexical structure, and binding/scope all in one pass. Renamed the language from QueryLite to DataScript for branding.

Rejected / modified.

Nothing major rejected at this stage. I changed my mind on the language name once.

Errors caught.

None caught here — the issues showed up later when the partner and I switched to Recipix.

Reflection.

Writing D1 alongside the model with 'explain it like the exam will ask' constraint forced me to actually engage with each decision instead of accepting boilerplate. Exam-driven prompting works.

Entry # 17 of 39 • Date 2026-05-04 • Phase Implementation (DataScript) • AI tool Claude Sonnet 4.6

Goal.

Build the DataScript lexer and parser in Python and produce sample programs.

Prompt.

Turns covering: setup, write lexer, write parser, write the three valid sample programs and the five invalid ones. Mostly "evet"/"devam"-style moves through a TodoWrite-tracked plan.

Accepted.

Produced lexer.py, parser.py, ast_nodes.py, main.py, three valid examples (query_basic.ds, func_aggregate.ds, if_filter.ds), five invalid examples covering distinct error classes. The full pipeline (`python main.py --dump-ast file.ds`) worked end-to-end.

Rejected / modified.

Nothing rejected at the time.

Errors caught.

Self-caught the obvious bugs as it went. Did not catch any spec violations because at that point I was still inventing the DataScript spec on the fly.

Reflection.

TodoWrite-tracked multi-step implementation in Claude Code is much smoother than back-and-forth in a chat window. The model held the plan in its head and ticked items off.

Entry # 18 of 39 • Date 2026-05-05 • Phase Verification + git • AI tool Claude Sonnet 4.6

Goal.

Verify everything works, then push to GitHub.

Prompt.

Several short turns: "d2 ye geçmeden önce yaptıklarının doğru ve pdfle örtüştüğünü doğrulamak için nelere bakmalıyım" (what do I check to verify before D2), "yes i did they all wroking correctly", "now i need to psuh this github", plus pasted git error output.

Accepted.

Walked through verification: each Part-1 deliverable maps to a checklist item, each example program runs cleanly. For the git push: identified the rejected-because-fetch-first error, walked through `git pull --rebase` then `git push` correctly.

Rejected / modified.

Nothing rejected.

Errors caught.

Caught a `git commit` bug where my single-quoted commit message was being misparsed by PowerShell (` apostrophe in the message was breaking the quote). Suggested switching to double quotes.

Reflection.

Git is the kind of tool where the model is much faster than me at recognizing standard error patterns. Outsourcing git troubleshooting to the model saves real frustration.

Entry # 19 of 39 • Date 2026-05-07 • Phase Pivot from DataScript to Recipix • AI tool Claude Sonnet 4.6

Goal.

Switch the project entirely from DataScript to Recipix (my partner and I changed our mind on the domain).

Prompt.

"we change our decision to use another dsl i uploaded the file explained our design after that we build that pl" — uploaded the recipix_v4_1_spec.md. Then "ops i forget to say i will build lexer and my partner implement parser firstly i should push new design file to github also deleted old folders to prevent any conflict for partner." Then "i deleted fodlers manuelly is this enough for removing from repo".

Accepted.

Read the new spec. Confirmed the scope split: I own the lexer, partner owns the parser, joint integration. Walked me through the cleanup: `git rm` for each deleted file, then `git add -A`, then commit and push.

Rejected / modified.

Caught my mistake — manually deleting files from the local folder does NOT remove them from the git repo. I needed `git rm` or `git add -A` to stage the deletions.

Errors caught.

The "manual delete is not enough for git" lesson would have caused a confusing repo state — old DataScript files would still be in the repo, conflicting with my partner's parser.

Reflection.

Switching domains a day before the deadline is the kind of decision that's only safe with strong tooling. The model handled the migration cleanly.

Entry # 20 of 39 • Date 2026-05-07 • Phase Lexer build • AI tool Claude Sonnet 4.6

Goal.

Build the Recipix lexer (with the partner-shared tokens.py contract).

Prompt.

"ok now lets go build phase for lexer but we are 2 persons group at final stage we should entegrate parser and lexer"

Accepted.

Started with the right move — said "for clean integration, both you and your partner need to agree on token types before you start. Your partner's parser will import from your lexer. So the first thing we build is the shared token contract." Created tokens.py first as the contract, then lexer.py implementing it. Walked through the trickiest parts: ``C` as one Unicode token, two-token quantity literals, `` line comments, large keyword set.

Rejected / modified.

Nothing rejected.

Errors caught.

Self-caught a Python `SyntaxError: unterminated string literal` in my one-liner test command — the inline quotes were conflicting. Wrote a separate test file instead, cleaner.

Reflection.

The "publish the contract before either side starts" instinct was the right call for pair work. Without it, my partner's parser would have imported a moving target.

Entry # 21 of 39 • Date 2026-05-08 • Phase Lexer ↔ partner's parser integration • AI tool Claude Sonnet 4.6

Goal.

Pull the partner's parser, integrate with my lexer, and verify end-to-end.

Prompt.

"ok berks says he done his part and combine lexer and parser we should check this it is really done firstly pull from gitub and control architecture" — then pasted git diff output.

Accepted.

Read every file the partner had added (parser.py, ast_nodes.py, errors.py, pretty.py, __main__.py, README, test_parser.py + 8 .rcx fixtures). Checked the architecture matched my mental model: lexer reads .rcx file → tokens → parser consumes tokens → AST → `--dump-ast` prints. Confirmed the partner's code was solid.

Rejected / modified.

Caught a real bug while integrating: the lexer accepts `200g` (no separator) but spec Decision #7 requires this to be a lexer error. The bug was in `\_scan\_number()` — after scanning the digits, the next-token call was swallowing whitespace before checking for the missing separator. Fix was to peek at the immediate next char inside `\_scan\_number()` itself before the whitespace skip.

Errors caught.

The 200g bug. Spec Decision #7 was the source of truth; the model went and re-read the decision before fixing.

Reflection.

Decision-pinned bugs ('this must follow Decision #N') are easier for the model to fix correctly than vague-feel bugs. Pin to the spec when you can.

Entry # 22 of 39 • Date 2026-05-08 • Phase Full pipeline verification • AI tool Claude Sonnet 4.6

Goal.

Run the full lexer→parser→AST pipeline end-to-end on real sample programs.

Prompt.

"ok for now we need integrate whole system how can i do that architecture shoould be like that lexer reads recipe files and generate tokens parser parse these tokens" — then pasted the AST dump output.

Accepted.

Wrote a clean root-level main.py wrapping the partner's `\_\_main\_\_.py`. Test suite ran sample1.rcx (pancakes with scale), sample2.rcx (smoothie with substitute), sample3.rcx (deferred type-error), and the five invalid fixtures. Every construct from the spec (QuantityLit, RecipeDecl, FunctionDecl, IngredientDecl, StepDecl, at/for modifiers, RepeatStmt, ScaleCall, KwArg) appeared in the AST dump correctly.

Rejected / modified.

Nothing rejected.

Errors caught.

Caught a confusion of mine: I asked "is it storing output anywhere" — the model correctly asked for clarification rather than guessing whether I meant "no output appeared" or "where is the AST saved to disk."

Reflection.

When I asked an ambiguous question, the model asking for clarification (instead of guessing wrong) saved a turn.

Entry # 23 of 39 • Date 2026-05-08 • Phase Polishing / pipeline cleanup • AI tool Claude Sonnet 4.6

Goal.

Clean up the lexer error message handling, dedupe the two tokens.py files, fix `python -m recipix` invocation.

Prompt.

A series of small turns: "i changed get_tokens for clean message for 200g or any other issues for lexer error can u check it", "now i want to run whole project but there is 2 tokens file what should be use", "why i cant run the python -m recipix bad.rcx", "i want to run whole project python main.py <file.rcx> this command".

Accepted.

Walked through each cleanup: (a) the lexer error path now exits cleanly via `sys.exit(str(e))` instead of crashing with a traceback; (b) deleted the duplicate `src/tokens.py` (kept the one in

`src/recipe/`); (c) fixed the `\_\_main\_\_.py` import so `python -m recipix` works; (d) made `python main.py <file.rcx>` the canonical entry point.

Rejected / modified.

Nothing rejected.

Errors caught.

Found that the README was telling users to run `python main.py <file.rcx>` but ``<file.rcx>`` was missing in some examples. Fixed the README in the same pass.

Reflection.

Multiple small turns each focused on one cleanup item is faster than one giant 'clean everything up' turn. The model loses precision when given a sweeping prompt.

Entry # 24 of 39 • Date 2026-05-08 • Phase Wrap up + commit • AI tool Claude Sonnet 4.6

Goal.

Confirm everything is done for D2 (implementation), commit and push, and clarify what's next.

Prompt.

Several wrap-up turns: "now tell me how this project works lexer doing what and it is job of project how it called also same for the parser and test files", "where is the storing tokens after lexer part to send them to the parse", "okey now u saying everythin is done about implementing part other parts are next", "ok lets push them everything", plus a final post-deadline ebnf-question turn.

Accepted.

Walked through the full architecture for the May-14 exam answer (lexer reads chars → tokens; tokens stored in a Python list returned by `tokenize()`; parser consumes that list with index-based lookahead; produces AST). Confirmed D2 is done. Helped commit and push everything cleanly.

Rejected / modified.

Nothing rejected.

Errors caught.

Final spec audit: I asked "assignment says need ebnf when designing dsl?" — the model went back to the handout PDF and the spec, confirmed yes, EBNF is required, and confirmed our spec.md *had* one (§13). Avoided me re-doing a deliverable I'd already met.

Reflection.

Closing a long session with 'walk me through the whole thing again as if for an exam' is the single best way to lock in the material before the written exam. The model is patient about being asked to re-explain things.

Session 3 — Tester Agent (Claude.ai Web)

Source: recipix_chat_log.md

A separate Claude.ai web session set up as a tester agent. Goal: run the lexer + parser against the spec, find any disagreements, produce a written report. May 8, 2026 — ten exchanges, condensed below into eight substantive entries (the GitHub-fetch-blocked turns are folded in).

Entry # 25 of 39 • Date 2026-05-08 • Phase Tester role setup • AI tool Claude.ai web (Sonnet)

Goal.

Set up a tester-agent Claude that reviews our lexer + parser against the assignment and design files.

Prompt.

"You are the tester agent dont forget i ll give you my programming language as semester project part 1 we are two persons group and developed project but not sure it is really correct i ll upload assignment pdf and design files include rules and data related 2 our developed domain specific language and then project files"

Accepted.

Acknowledged the role and the upload plan. Asked for assignment PDF, design files, project files in that order.

Rejected / modified.

Nothing rejected.

Errors caught.

None.

Reflection.

Same role-priming pattern as the professor session — set the stance up front, in this case "tester not collaborator," and the model behaves accordingly.

Entry # 26 of 39 • Date 2026-05-08 • Phase Network workaround • AI tool Claude.ai web (Sonnet)

Goal.

Get the tester to read project files from GitHub, since file uploads were failing.

Prompt.

"i ll give you github links because i cant upload file claude giving error when i try to upload file" → "https://github.com/ismail-kabak/CSE341_Semester_Project_Part_1 check firstly cse 341 project handout file and then recipix v4_1 spec md (our pl rules) and finally test the whole project"

Accepted.

Tried web_fetch on the GitHub URLs; all returned PERMISSIONS_ERROR. Honestly reported the block: "Claude's network policy only allows fetching URLs that come directly from you or from search results. GitHub's API and raw content URLs are being blocked." Offered three workarounds (raw URLs, paste contents, public repo).

Rejected / modified.

The whole GitHub-fetch approach was rejected by the network policy. I switched to file uploads instead.

Errors caught.

Caught the network restriction immediately and reported it cleanly instead of timing out silently.

Reflection.

The model being upfront about a tool failing — "I tried, it returned this error, here are alternatives" — is much better than it pretending to fetch and hallucinating contents.

Entry # 27 of 39 • Date 2026-05-08 • Phase Independent test run • AI tool Claude.ai web (Sonnet)

Goal.

Run an independent test battery against the lexer + parser to find any disagreements with the spec.

Prompt.

Uploaded all source files (parser.py, lexer.py, ast_nodes.py, tokens.py, errors.py, pretty.py, __main__.py, __init__.py, main.py).

Accepted.

Wrote a custom run_test.py with 10 test sections (66 cases): spec sample programs, lexer rules, operator precedence, substitute rules, control structures, top-level restrictions, edge cases, potential bug checks, AST node correctness assertions, deferred-issue probes. All 66 passed. AST shape verified explicitly: SubstituteCall.original_name and replacement_name are str (not Expr); quantity_of(...) produces QuantityOf node (not FunctionCall); QuantityLit has correct number/unit fields; CompareOp produces correct op string; BinaryOp + is left-associative.

Rejected / modified.

Flagged 5 deferred issues (not bugs): scale/substitute call-site enforcement is parser-level only, return inside step body accepted, zero-arg call disambiguation, empty recipe accepted, `5 pinch` parses without normalization. Each was correctly diagnosed as Part-2 type-checker work, not Part-1 parser bugs.

Errors caught.

Zero parser bugs found. Five deferred-to-Part-2 items correctly identified and explained.

Reflection.

Building 66 fresh tests beyond what we'd already written gave real confidence the parser handles the corners. AI is good at the 'volume side' of testing.

Entry # 28 of 39 • Date 2026-05-08 • Phase Run our own tests too • AI tool Claude.ai web (Sonnet)

Goal.

Run our own existing test files so the comparison is honest (not just AI tests against AI code).

Prompt.

"i had some test files but i want to test it yourself after that i ll upload my test files for there is any issue" — uploaded test_parser.py, test_lexer.py, plus 8 .rcx fixtures.

Accepted.

48/48 parser tests passed. 12/12 lexer tests passed. Confirmed `°C` tokenized correctly, all 4 lexer error cases (`@`, unterminated string, lone `°`, glued `200g`/`1.5kg`) rejected with correct messages.

Rejected / modified.

Caught a small setup issue — running `python -m unittest test\_parser` fails with `ModuleNotFoundError`. The correct command is `PYTHONPATH=src python -m unittest test\_parser -v`. Recommended documenting it in the README or adding a confest.py.

Errors caught.

The PYTHONPATH issue. Worth a one-line README addition.

Reflection.

A real-life integration nit (working command vs. broken command) showed up only when the model tried to run things from a different working directory. I would never have caught this — I always run tests from the repo root.

Entry # 29 of 39 • Date 2026-05-08 • Phase Summary report • AI tool Claude.ai web (Sonnet)

Goal.

Produce a single summary report covering what passed, what didn't, what's deferred.

Prompt.

"ok make a summary including bugs or any issue missing things"

Accepted.

Tried to render a visual summary widget, timed out after 4 minutes, fell back gracefully to a text summary. Final numbers: 60 user tests + 66 independent tests = 126 total, all green. 0 actual bugs. 5 deferred-to-Part-2 items. 1 README setup nit. Tabulated each spec decision (#7, #17, #20, #23, #34, #35, §8, §9, §7) against pass/fail; all green.

Rejected / modified.

Nothing rejected.

Errors caught.

None.

Reflection.

The widget timeout was annoying but the graceful fallback to a text table was the right move. AI tooling fails sometimes; what matters is whether the fallback works.

Entry # 30 of 39 • Date 2026-05-08 • Phase Part-2 risk assessment • AI tool Claude.ai web (Sonnet)

Goal.

Sanity-check whether the 5 deferred issues will make Part 2 harder.

Prompt.

"for 5 deferred issues affect part 2 development phase become harder or they can be fixable during part 2 development"

Accepted.

Walked through all 5 with effort estimates: (A) scale/substitute call-site enforcement → trivial, one instance check; (B+E) return context/position → easy, you're visiting that node anyway for return-type checking; (C) zero-arg call disambiguation → already free, your symbol table resolves it; (D) pinch normalization → trivial, ignore the number field for pinch unit. Closed with a useful framing: Part 1 enforces what the grammar can express; Part 2 enforces what meaning requires. The hard part of Part 2 is the dimensional type system itself, not these deferred items.

Rejected / modified.

Nothing rejected.

Errors caught.

None caught (none to catch).

Reflection.

Effort estimates ("trivial / easy / free / one line") are more useful than abstract reassurance. Made me confident I could ship Part 2 in time.

Entry # 31 of 39 • Date 2026-05-08 • Phase Final readiness sign-off • AI tool Claude.ai web (Sonnet)

Goal.

One-line verdict: is Part 1 done correctly?

Prompt.

"ok as a last word part 1 implementation done right ?_"

Accepted.

"Yes, Part 1 is done correctly." Then concrete defense: every spec decision implemented, all 114 tests pass, AST shapes match spec exactly, three sample programs from §11 all parse. The only fix is the PYTHONPATH README note. Closed with exam-targeting hint: be ready to defend Decisions #34, #35, #17 since those are the spec's "signature decisions."

Rejected / modified.

Nothing rejected.

Errors caught.

None.

Reflection.

The "exam will ask about these decisions, your code defends them" framing was the most useful thing the tester said. It tied the testing back to what the May 14 exam actually grades.

Entry # 32 of 39 • Date 2026-05-08 • Phase Documentation • AI tool Claude.ai web (Sonnet)

Goal.

Save the entire chat as a documentation artifact for the project.

Prompt.

"create chat log include all chat each prompt and answer dont ignore any of them list is so long but need them everything to documentation"

Accepted.

Produced the recipix_chat_log.md file with all 10 exchanges, plus a files-uploaded table and a final test-totals table.

Rejected / modified.

Nothing rejected.

Errors caught.

None.

Reflection.

Capturing chat logs as artifacts at the end of a session is the kind of housekeeping I'd skip if I had to do it manually. The model produces it in 10 seconds.

Session 4 — Code Reviewer (Claude.ai Web)

Source: recipix_chat_log__1_.md

Another Claude.ai web session, this time framed as a Part-2 risk-focused code review. Goal: catch issues that won't break Part 1 but will block or complicate Part 2. May 8, 2026 — eight substantive turns.

Entry # 33 of 39 • Date 2026-05-08 • Phase Reviewer role setup • AI tool Claude.ai web (Sonnet)

Goal.

Set up a code-review-focused Claude that reads the source files and finds issues that affect Part 2 implementability.

Prompt.

"we developed a our domain spesific programming language and ur role code reviewing check this files and detect is there any issue effecting part2 implementing will be hard i ll uplaod assignment pdf and desing file about the dsl"

Accepted.

Acknowledged the role, asked for the files.

Rejected / modified.

Nothing rejected.

Errors caught.

None.

Reflection.

Setting up the reviewer with a Part-2-risk frame instead of a generic "review my code" frame produced more targeted findings later. Worth doing.

Entry # 34 of 39 • Date 2026-05-08 • Phase Reviewer mistake / correction • AI tool Claude.ai web (Sonnet)

Goal.

Have the reviewer review the actual *code* — not the spec.

Prompt.

Uploaded the handout PDF and the v4.1 spec markdown, expecting the model to wait for source code files. The model jumped in and reviewed the spec markdown as if it were code. Then I corrected: "nope i didnt upload my code files".

Accepted.

Acknowledged the mistake without making excuses: "You're right, my apologies! I reviewed the spec document (recipix_v4_1_spec.md) as if it were the code — but you said you'd upload code files for review." Asked for the actual source files.

Rejected / modified.

I rejected the spec-as-code review and got a clean restart.

Errors caught.

The model misread which files were code and which were spec — the file extensions should have made that obvious. Worth knowing the model can make that mistake.

Reflection.

The model didn't double-down or argue. Said "you're right" and reset. That's the response I want when an AI is wrong.

Entry # 35 of 39 • Date 2026-05-08 • Phase Part-2-risk code review • AI tool Claude.ai web (Sonnet)

Goal.

Read the actual source files and produce a list of Part-2-risk issues organized by severity.

Prompt.

Uploaded all source files (parser.py, lexer.py, ast_nodes.py, tokens.py, errors.py, pretty.py, __main__.py, __init__.py, main.py). "ok check these files is there any issue to affect part 2 implementation maybe become harder or overengineering code parts chec"

Accepted.

Produced an 11-issue review organized by severity: 3 critical (will directly break Part 2), 4 major (will make Part 2 harder), 4 minor (style/clarity). Each issue: file:line, what's wrong, why it hurts Part 2, concrete fix.

Rejected / modified.

Nothing rejected.

Errors caught.

Critical findings the tester agent had not surfaced: (1) parser.py line ~619 — zero-arg calls always produce FunctionCall, even when the name is a recipe — Part-2 type checker would look up the wrong table; (2) FunctionDecl.body is a plain list with no enforcement that `return` is the last and only statement — needs to be enforced at parse time, not deferred; (3) `\_parse\_step\_decl` has a greedy `for`-expr parse with no termination guard — `for ... at ...` could swallow the body brace.

Reflection.

The Part-2-risk frame surfaced different bugs than the tester's "does it match the spec" frame. Same code, two different lenses, two different bug lists. Worth running both.

Entry # 36 of 39 • Date 2026-05-08 • Phase Generate fix prompt • AI tool Claude.ai web (Sonnet)

Goal.

Produce a copy-pasteable prompt I can hand to a coding agent to fix all 11 issues.

Prompt.

"ok now make prompt to give coder agent to fix these issue"

Accepted.

Produced a structured agent prompt with all 11 fixes laid out: file path, exact location, what to change, what error message to raise, what AST changes to make. Each fix self-contained.

Rejected / modified.

Nothing rejected at this stage. I had a downstream issue with copy-paste though.

Errors caught.

None caught — the prompt was clean.

Reflection.

Asking the model to produce a fix prompt for another model is a useful pattern. Each model's strengths get used (this one for review, the next one for execution).

Entry # 37 of 39 • Date 2026-05-08 • Phase File-format request • AI tool Claude.ai web (Sonnet)

Goal.

Get the fix prompt in a downloadable file rather than as chat text.

Prompt.

"i cant paste this directly so create a docx file or md file to copy paste"

Accepted.

Produced recipix_critical_fixes.md as a downloadable file with the 3 critical fixes only (matched my next instruction to focus on critical only).

Rejected / modified.

Nothing rejected.

Errors caught.

None.

Reflection.

When chat text gets long, asking for a file is faster than scrolling. Cheap habit, big payoff.

Entry # 38 of 39 • Date 2026-05-08 • Phase Scope reduction • AI tool Claude.ai web (Sonnet)

Goal.

Reduce the fix list from all 11 to just the 3 critical ones (Part-1 deadline pressure).

Prompt.

"but focus critical part for now"

Accepted.

Re-generated the file focused only on the 3 critical fixes: AmbiguousCall node for zero-arg calls (4 numbered steps), return placement enforced at parse time (early-return → ParseError, missing-final-return → ParseError), greedy-for guard in step parsing.

Rejected / modified.

The other 8 issues were dropped from the immediate-action list (still in the chat for Part 2).

Errors caught.

None caught at this stage.

Reflection.

Triage by severity is what saved the deadline. The model's severity tags (critical/major/minor) made the cut trivial.

Entry # 39 of 39 • Date 2026-05-08 • Phase Final review after fixes • AI tool Claude.ai web (Sonnet)

Goal.

Verify all 3 critical fixes were correctly applied.

Prompt.

"i fixed smt check this version and if there is any critical bugs or deferred issue which not preventing run correctly for part1 this means done if it has tell me" — uploaded updated source files.

Accepted.

Walked through each fix:  Fix 1 — AmbiguousCall is in ast_nodes.py, imported in parser.py line 43, zero-arg branch at line 654 returns it, pretty.py has the dump handler.  Fix 2 — return enforcement done.  Fix 3 — greedy-for guard in place.

Rejected / modified.

Nothing rejected — fixes were correct.

Errors caught.

No new bugs found. Part 1 was confirmed shippable.

Reflection.

Fix → re-review → confirm is a 3-turn loop. Doing it after every fix keeps regressions from accumulating. Trying to batch fixes and re-review at the end always backfires.